

FLEX+: moving the ceiling

An experiment branch. The ladder shape is fixed at six exits split at depth two, as the paper reports it; the question is whether the architectural ceiling above it can be raised without touching either.

1. What the ceiling is made of

The paper's ceiling is 38.33% counted. It is not a routing result and no better router can move it: it is what a frame costs when every tile already sits on the shallowest rung it is allowed. At a 0.2 dB budget the deployed system delivers 36.81% of that 38.33%, which is 96% of the ceiling, so the allocation is very nearly finished and the ceiling is the whole of what is left.

Decomposing that floor decides the experiment by arithmetic rather than by taste. A frame with every tile on the shallowest rung costs 0.6088 of a released decode, and one part of it is large enough to matter.

part of the floor	cost	share
shared trunk, 4 blocks, always, full frame	0.2981	49.0%
tiled trunk, 2 blocks	0.1491	24.5%
upsample, always, full frame	0.0816	13.4%
adapter at the shallowest exit	0.0464	7.6%
head, with its halo	0.0240	3.9%
seam repair	0.0095	1.6%

Table 1. Where the ceiling's floor goes. Frame-relative, halo on the head, from flexuf.cost on the paper's configuration.

Halving the shared trunk puts the ceiling at 54.0% and quartering it at 61.5%. Halving anything else leaves it below 47%: the upsample gone entirely would give 45.2%, the adapter 42.6%, the head 40.9%. There is one lever, and the target this branch was opened for -- 55% at a 0.2 dB budget -- is reachable only through it.

2. Why the exit ladder cannot reach it

The four blocks below the split run before the frame is tiled, on every pixel, whatever depth any tile chooses. Depth is one axis and it does not intersect them. Lowering the split depth would, and that is the move that is closed: it is also what creates the seam the paper spends a section on. So the ceiling needs a second axis, applied to the stem, that does not tile it.

2.1 What the literature offers

Width. Slimmable Networks (Yu et al., ICLR 2019) trains one network executable at several channel widths with shared weights and per-width normalisation, and reports each width close to a network trained at that width alone. Dynamic Slimmable Network (Li et al., CVPR 2021) makes the width input-dependent with a gate. SLEXNet (ACM TECS, 2024) combines slimmable widths with early exits, which is the pairing here. Cost falls with the square of the width.

Resolution. RANet (Yang et al., CVPR 2020) routes easy inputs through a low-resolution path and exits them early, keeping a high-resolution path for the rest. Half resolution is a quarter of the cost, the same scaling as half width.

Spatial sparsity. Dynamic Convolutions (Verelst and Tuytelaars, arXiv:1912.03203) learn a spatial mask and execute convolutions only where it is set; focused convolutions (arXiv:2310.07782) do the same for pretrained networks. Cost falls linearly with the active fraction, which is weaker, but it is the only one of the three that changes no tensor shape and therefore adds no tile boundary.

Width is the axis this branch follows, on instruction, and it is also the one with the strongest scaling. It is applied to the stem as a whole rather than per tile, so it creates no seam either.

3. The stem does not tolerate being cut

Before building anything, the cheapest question: can the stem simply be made smaller? Two training-free degradations, measured at the shallowest exit against the released decoder on twelve frames.

stem	q0	q32	q63
half	+4.27	+9.77	+13.94
drop 0.25	+0.50	+1.20	+2.21
drop 0.5	+1.14	+2.09	+3.29
drop 0.75	+1.96	+2.79	+4.07

Table 2. Extra dB from degrading the stem, untrained, above what the shallowest exit already costs. The budget this branch is aiming at is 0.2 dB in total.

Every entry is outside the budget, most of them by an order of magnitude. A stem at half resolution spends forty times the budget at the lowest rate and seventy at the highest. Dropping a quarter of the channels -- the mildest cut available -- costs two and a half times the whole allowance at the lowest rate and eleven times at the highest.

The deeper trunk repairs a little of the damage, and only a little: at a quarter of the channels dropped the deepest exit is 0.10 dB better off than the shallowest at the lowest rate and 0.15 dB at the highest, against a penalty of 0.50 and 2.21. It is not a mechanism to build on.

This is conclusive about cutting and says nothing about slimming. An untrained slice is exactly what the slimmable literature exists to improve on, so the result is a floor rather than an answer.

4. A narrow stem, trained to match the full one

The four stem blocks are replaced by a module of width w with a 1×1 projection either side, so everything downstream still sees the same channel count and needs no change. Its cost is w squared of the original plus the two projections. The residual is around the whole module and the output projection is zero-initialised, so at step zero it is exactly the decode that skips the stem -- a known starting point rather than noise.

It is trained to reproduce the full stem's output feature, with the encoder, the entropy model, the rest of the trunk, the exits and the head all frozen. The target is therefore a pure regression and any quality change is attributable to the stem alone.

The width sweep is still training. This section is written and its table is empty until flexplus/narrow_eval.py has run; the document is built from the result files, so it fills itself in.

5. Open

Whether a width that fits the budget exists is the whole question and it is not answered here yet. If one does, two things follow that this experiment does not test: the narrow stem has to be trained jointly with the ladder rather than fitted to a frozen target, and a single fixed width is a weaker design than a gate that chooses one per frame, which is what Dynamic Slimmable Network is for.